

Sección 1 — Lectura

Plataforma web para el aprendizaje de idiomas

Documentación técnica: sistema de lectura y pipeline de PLN

Español · Inglés · Alemán | React + Vite · Python · spaCy (de/en/es) · FreeDict · opus-mt / Gemini · wordfreq

Documento de referencia para la memoria de tesis y el auto-aprendizaje. Recoge las librerías y modelos empleados, la arquitectura, los pipelines, los errores encontrados y su solución, y las ventajas y desventajas de cada decisión de implementación.

Índice

1. Introducción y objetivos
2. Arquitectura general
3. Frontend (React + Vite)
4. Pipeline de PLN offline (Python)
5. Traducción por oración de libros (LLM vs MT)
6. Errores encontrados y soluciones
7. Ventajas y desventajas de cada implementación
8. Librerías, modelos y recursos
9. Insights y metodología
10. Estado actual y trabajo futuro
11. Glosario técnico

1. Introducción y objetivos

El proyecto es una plataforma web para aprender español, inglés y alemán, concebida como trabajo de tesis en **matemática algorítmica**. La aportación central de la tesis no es la herramienta en sí, sino el **modelado matemático y algorítmico** que la sustenta (procesos estocásticos, modelos de frecuencia léxica, reglas morfológicas, álgebra lineal, teoría de grafos y de números); el sitio es el medio donde esas herramientas viven y se demuestran en funcionamiento.

La plataforma se organiza en cuatro vertientes: (1) **lectura**, (2) **repaso de vocabulario** con repetición espaciada (SM-2 en producción, Leitner como cadena de Markov por simulación), (3) **gramática** generada con PLN sobre los propios textos, y (4) **juegos** de palabras. Este documento cubre la **vertiente de lectura** y su **pipeline de PLN** en los tres idiomas de estudio: alemán e inglés (traducidos al español) y **español como lengua de estudio para nativos** (libros difíciles con glosado de voces raras).

Alcance de lo construido

- Sección de **Lectura** completa: biblioteca por idioma/nivel, lector con traducción por palabra y por frase, bolsa de palabras persistente y seguimiento de progreso.
- **Tres idiomas de estudio**: alemán e inglés (con traducción al español) y **español como lengua de estudio para nativos**, con libros difíciles glosados (definiciones de voces raras, no traducción).
- **Pipeline de PLN offline** en Python que ingiere libros de dominio público, los procesa con spaCy (modelo por idioma) y genera el JSON que consume el frontend (traducción por palabra, por frase, y glosado monolingüe del español).
- **Idioma de estudio global** (contexto React persistido) que gobierna todas las secciones; bolsa filtrada por idioma; **perfil exportable/importable** como JSON sin servidor.
- **Motor de lógica pura** con pruebas unitarias (Vitest), germen del núcleo matemático.
- Mejoras al **juego Codenames**: extracción del generador congruencial a un módulo puro, semilla que codifica idioma y nivel, y optimización de recursos.

2. Arquitectura general

El sistema se estructura en tres piezas claramente separadas, siguiendo el diseño de la tesis: un pipeline offline que hace el trabajo pesado, un motor algorítmico puro, y un frontend que solo presenta e interactúa.

Pieza	Tecnología	Responsabilidad
Pipeline offline	Python (spaCy, FreeDict, transformers)	Ingiere textos de dominio público, calcula segmentación, lematización y traducción; exporta JSON. Se ejecuta una sola vez, fuera de línea.
Motor algorítmico	JavaScript puro (+ Vitest)	Lógica sin efectos secundarios: bolsa de palabras, generación determinista del tablero, progreso. Testeable en aislamiento.
Frontend	React 19 + Vite 8 + react-router 7	Consume el JSON precalculado y gestiona presentación e interacción. No ejecuta procesamiento pesado en vivo.

INSIGHT — Principio de diseño: “sin APIs en vivo”

La aplicación en ejecución es **determinista y sin red**: todo el estado vive en `localStorage` y todos los datos (textos, traducciones, léxico) son JSON estático precalculado. El coste computacional (PLN, traducción) se paga una vez en el pipeline, no en cada visita.

Matiz importante: usar un LLM (Gemini) o un modelo de MT para traducir un libro es un **paso offline de una sola vez** que produce JSON estático; no rompe ese principio, porque la app nunca llama a una API en tiempo de ejecución.

Flujo de datos: textos de dominio público (Project Gutenberg) -> pipeline Python (spaCy + diccionarios) -> archivos JSON en src/data/ -> frontend React que los carga y renderiza. El estado del usuario (bolsa, progreso) se guarda en el navegador.

3. Frontend (React + Vite)

3.1 Stack y versiones

Librería	Versión	Para qué
React	19.2	UI declarativa por componentes.
Vite	8.0	Bundler y servidor de desarrollo (HMR). Rollup/Oxc por debajo.
react-router-dom	7.18	Enrutado por secciones y URLs profundas.
Vitest	4.1	Pruebas unitarias del motor puro.
ESLint	9	Análisis estático.

3.2 Enrutado y secciones

App.jsx pasó de ser el router del juego a un router de **plataforma** con BrowserRouter. Rutas: / (menú), /juegos/* (Codenames), /lectura (biblioteca), /lectura/:idioma/:nivel/:id (lector) y /bolsa. Para que las URLs profundas no den 404 en Netlify se añadió public/_redirects con /* /index.html 200 (fallback SPA).

3.3 Modelo de datos

Cada **lectura** es un JSON con metadatos y el cuerpo alineado por frase entre idiomas. Las lecturas ya no tienen que ser trilingües: pueden ser de+es, en+es, o **solo es** (libros de estudio para nativos). La regla de idiomasDisponibles es: de/en son idiomas de estudio si tienen cuerpo; el español lo es **solo cuando es el original** (no hay de/en) — en las demás lecturas actúa como idioma de traducción. Los libros añaden campos libro/parte/partes para agruparse.

```
{
  "id": "mercado-01", "nivel": "principiante", "autor": "...",
  "titulo": { "es": "El mercado", "en": "The market", "de": "Der Markt" },
  "fuente": "...procedencia y URL de dominio público...",
  "cuerpo": {
    "de": ["Sie kauft Äpfel, Brot und Käse.", ...], // frase i
    "es": ["Compra manzanas, pan y queso.", ...] // frase i (alineada)
  }
}
```

El **léxico** (traducción por palabra) es un mapa idioma:forma -> { lemma, es }. Se busca por la forma superficial normalizada (minúsculas, sin puntuación), conservando acentos y umlauts. En las entradas es: el campo es no es una traducción sino una **definición en español moderno** de una voz rara.

```
{ "de:frau": { "lemma": "Frau", "es": "mujer" },
  "en:mice": { "lemma": "mouse", "es": "ratón" },
  "es:adarga": { "lemma": "adarga", "es": "escudo de cuero, ovalado..." } }
```

La **bolsa de palabras** es única y persistente; cada entrada guarda id, lang, surface, lemma, traducciones, origen, addedAt. Está diseñada para que el motor de repaso (Sección 2 de la tesis) le agregue después su estado (caja de Leitner, factor SM-2).

3.4 Motor de lógica pura + persistencia

Se separó el **núcleo puro** (sin DOM, React ni localStorage) de la persistencia. Esto es lo que la tesis valora como “motor testeable” y de paso elimina duplicación de código.

- `engine/board.js`: generador congruencial lineal (LCG), Fisher-Yates, composición/parseo de semilla y generación determinista del tablero.
- `engine/bolsa.js`: alta/baja en la bolsa. Regla de la tesis: **una palabra ya presente conserva su estado** (no se reinicia el progreso).
- `engine/progreso.js`: lecturas completadas.
- `engine/almacenamiento.js`: adaptador delgado (impuro) sobre localStorage.

Pruebas con **Vitest** (17 casos): determinismo del tablero (misma semilla -> mismo resultado, distribución 9/8/7/1), la regla de no-reinicio de la bolsa, normalización, etc.

INSIGHT — El LCG como puente con la teoría de números

El juego sincroniza los tableros de ambos equipos con un **generador congruencial lineal** sembrado por un hash de la semilla. La semilla se rediseñó a formato XX-YYYY, donde XX codifica idioma y nivel del vocabulario; así el capitán solo introduce la semilla y el sistema deduce el diccionario. Esto conecta con aritmética modular y teoría de números, tal como plantea la tesis.

3.5 Experiencia de lectura

- **Tokenización** por expresión regular Unicode (`\p{L}...`) que separa palabras clicables de la puntuación, conservando el texto original.
- **Traducción por palabra**: al tocar una palabra se busca en el léxico y se muestra un popup con la traducción al español y un botón para añadirla a la bolsa.
- **Traducción por frase**: un marcador discreto en el margen de cada frase revela su traducción alineada. Se eligió un objetivo **táctil** (no hover) para que funcione en móvil; el hover es solo una mejora en escritorio.
- **Libros**: las partes de un libro colapsan en una sola tarjeta con progreso “N/10 partes”; el lector ofrece navegación anterior/siguiente y “finalizar y continuar”.
- **Bolsa y progreso** persistentes en localStorage; la biblioteca conserva idioma y nivel en la URL para no reiniciarse al volver de una lectura.

3.6 Carga de datos y peso del bundle

El catálogo carga todas las lecturas automáticamente con `import.meta.glob`, de modo que añadir lecturas (a mano o desde el pipeline) no requiere tocar código. El **léxico** (465 KB, 5.779 entradas) se importaba de forma estática e inflaba el bundle inicial; se pasó a **carga bajo demanda** con `dynamic import`, que Vite emite como chunk aparte cacheado, descargado solo al abrir una lectura. Bundle inicial: **1018 KB -> 694 KB** (gzip 328 -> 253 KB).

3.7 Idioma de estudio global, bolsa por idioma y perfil

Al crecer a tres idiomas de estudio hizo falta una noción transversal de **idioma de estudio actual**. Es un contexto React (`contexto/idiomaEstudio.jsx`) persistido en `idiomaEstudio.v1`, con un selector compartido presente en el hub, el Repaso y la Bolsa; las demás secciones (Juegos, Gramática) leen el mismo contexto para elegir su bloque de datos.

- **La bolsa no se parte por idioma**: sigue siendo un único almacén `bolsa.v1` (cada entrada ya lleva `lang` y su `id` es `lang:forma`); se **filtra al consumir**. Detalle importante: el Repaso conserva la bolsa completa en memoria y solo arma la sesión con el idioma activo — si persistiera la lista filtrada, borraría el resto de idiomas.
- **Perfil exportable/importable**: todo el estado del usuario vive en unas pocas claves de localStorage detrás de `almacenamiento.js` (única frontera de I/O). `exportarPerfil()` las serializa a un JSON

descargable y `importarPerfil()` las restaura validando la envoltura y cada valor; da copia de seguridad y portabilidad **sin servidor**.

- **Popup de palabra:** se rediseñó en columna (palabra, traducción/definición a todo el ancho, botón debajo) para que las definiciones largas del glosado español se lean bien y el botón nunca desborde.

4. Pipeline de PLN offline (Python)

El pipeline vive en `pipeline/` y convierte libros de texto plano en las lecturas JSON que consume el frontend. Está separado en dos pasos: **ingesta** de un libro (`procesar.py`) y **construcción del léxico** sobre todas las lecturas (`construir_lexico.py`).

4.1 spaCy: segmentación, tokenización, lematización

Se usa **spaCy** con un modelo por idioma: `de_core_news_md` (alemán), `en_core_web_sm` (inglés) y `es_core_news_md` (español). spaCy segmenta el texto en frases, tokeniza y asigna a cada token su lema, categoría gramatical (POS), rasgos morfológicos y relaciones de dependencia. El pipeline aprovecha las **frases** (para alinear traducciones) y los **lemas** (para buscar en el diccionario). `procesar.py` elige el modelo según el idioma declarado por cada libro en LIBROS, admite ingerir un **fragmento** acotado (`max_chars`, para obras enormes como el Quijote) y un marcador `fin` (para recortar índices al final, como en *Azul...*).

INSIGHT — Verbos separables del alemán vía la dependencia svp

El alemán parte muchos verbos: “...bereitet ... vor” es el verbo **vorbereiten** (preparar). spaCy etiqueta el prefijo con la dependencia svp (separable verb prefix) apuntando al verbo. El pipeline lo detecta y **reconstruye** el lema real: `vor + bereiten = vorbereiten`, y hace que tocar cualquiera de las dos partes remita a la misma entrada. Esto resuelve automáticamente un caso que al principio hubo que corregir a mano.

Es un buen ejemplo de la tesis: usar el **análisis de dependencias** para resolver una ambigüedad morfológica de forma algorítmica.

4.2 Traducción por palabra: diccionarios FreeDict por capas

La traducción por palabra es **determinista y offline**. Se usan diccionarios libres de **FreeDict** (formato TEI/XML) y una estrategia por capas en `traductor.py` (para el alemán; el inglés va directo por `eng-spa`, sin capas):

Capa	Método	Marca
1. Directo	deu-spa (36.744 entradas), por lema y por forma	—
2. Cadena	deu-eng (~380 mil) -> eng-spa (~59 mil) cuando deu-spa falla	(vía inglés)
3. Compuesto	cabeza del compuesto alemán (sufijo de >= 3 letras)	(en compuesto)

El diccionario `deu-spa` por sí solo cubría ~70% del vocabulario de un libro; le faltan muchas palabras comunes (`unruhig`, `hilfflos`, `bogenförmig`...). La **cadena por inglés** (FreeDict `deu-eng` es enorme) cierra la mayor parte del hueco, aunque a costa de dos saltos y algo de ruido; por eso se marca “(vía inglés)”. La **descomposición de compuestos** (la cabeza de un compuesto alemán es el último elemento: `Holztür -> Tür -> puerta`) recupera compuestos ausentes.

4.3 Cobertura de traducción y su evolución

La **cobertura** es el porcentaje de formas de palabra distintas que reciben traducción. Se midió con un diagnóstico que clasifica los fallos por categoría gramatical (`diagnostico.py`). Evolución sobre *Die Verwandlung*:

Cambio	Cobertura
FreeDict deu-spa + modelo pequeño (sm)	70%
Modelo mediano (md): mejor lematización (abhielt -> abhalten)	81%
Cadena deu-eng -> eng-spa	90%
Descomposición de compuestos con cabezas de 3 letras	92%

El ~8% restante son sobre todo **verbos fuertes** que spaCy no lematiza bien (hing, dasaß) y compuestos raros. Para **principiante e intermedio** se alcanzó el **100%**: el léxico se reconstruye desde una **semilla curada a mano** (`lexico.base.json`) que además de las entradas del pipeline incluye 49 casos rellenados manualmente (nombres propios marcados como tal, imperativos como Komm/gib, y verbos fuertes como schlief/aß).

NOTA — Fusión idempotente del léxico

`construir_lexico.py` reconstruye el léxico completo desde la semilla curada más el vocabulario de todas las lecturas, de modo que re-ejecutar el pipeline es **idempotente** y no arrastra entradas obsoletas. Al fusionar, **nunca sobrescribe** una entrada ya traducida con una sin traducción (una misma forma puede recibir distinto lema según el contexto).

4.4 Ambigüedad y overrides por lectura

Un léxico global (clave idioma:forma) no puede desambiguar formas que dependen del contexto: *nahm* es parte de **annehmen** (suponer) en una fábula, pero es **nehmen** (tomar) en otra novela; *an* es partícula separable o preposición según la frase. Es el **sub-problema de ambigüedad** que la tesis plantea explícitamente (una misma forma admite varios análisis).

Solución adoptada: cada lectura puede llevar un pequeño mapa `lexico` con overrides que el frontend **prioriza** sobre el léxico global. Para los textos curados de principiante/intermedio se detectaron los verbos separables (dependencia svp) inequívocos dentro de cada texto y se fijó a mano el lema y la traducción correctos, verificados frase a frase.

INSIGHT — Por qué un léxico global no basta

El límite es de fondo: la traducción por palabra correcta depende de la **posición del token**, no solo de su forma. Los overrides por lectura resuelven los casos curados; la solución general sería anotar el léxico **por token** en el pipeline (algo que spaCy ya permite) y renderizar con esos tokens en vez de tokenizar en el cliente. Es un refactor mayor, anotado como trabajo futuro.

4.5 Glosado monolingüe del español (lengua de estudio para nativos)

El español dejó de ser solo la lengua-puente: hay libros difíciles (Quijote y Buscón en fragmento; *Azul...* de Darío y los *Cuentos* de Quiroga completos) pensados para que un **nativo** amplíe vocabulario. Aquí “traducir” no tiene sentido: se **glosa** — una definición breve en español moderno — y **solo las voces poco comunes**: arcaísmos del Siglo de Oro (adarga, rocín, fazaña), léxico modernista (ánfora, cendal) y regionalismos rioplatenses (mensú, tacuara, jangada).

- **Detección de rareza**: offline, con la frecuencia general del español (escala Zipf de la biblioteca `wordfreq`): se extraen los lemas con Zipf bajo y ocurrencias suficientes, que forman la lista de trabajo a curar.
- **Definiciones curadas a mano** en `pipeline/glosas_es.json` (~190 voces). Los MT no sirven aquí (no hay lengua destino); el diccionario monolingüe curado es la única vía con calidad garantizada.
- **Casado robusto**: `construir_lexico.py` genera entradas `es:forma` casando por forma superficial o por lema, porque el lematizador español tropieza con arcaísmos y enclíticas (celada -> “celado”, trujeron, preguntóle).

- **Solo libros de estudio:** la rama es del léxico se aplica únicamente a lecturas cuyo cuerpo es solo español (en las trilingües, es es la traducción, no texto a glosar).

INSIGHT — El modelo de conocimiento hace el resto

No hace falta lógica nueva para que el sistema priorice las voces raras: el modelo de conocimiento de la Sección 2 ya estima $P(\text{conocer})$ con un prior logístico sobre la frecuencia Zipf del corpus. Para un nativo, las palabras corrientes salen con probabilidad altísima y las raras bajan — el repaso previo del Lector destaca exactamente las glosadas. La misma matemática sirve para los tres idiomas.

5. Traducción por oración de libros (LLM vs MT)

Los libros de alemán e inglés solo traen el texto original; darles traducción **por frase** exige una versión española **alineada 1:1** con las frases originales (la frase es $[i]$ corresponde a la i -ésima del original). Esa alineación es la restricción que manda en todas las opciones. Se implementaron dos vías. Los libros en español no llevan traducción por frase: son monolingües y el Lector oculta el marcador.

5.1 Vía LLM (Gemini) — máxima calidad, semi-manual

Flujo con validación de alineación: `exportar_frases.py` vuelca las frases numeradas + un prompt estricto (“una línea por oración, no unir ni dividir”); se pega en Gemini; `importar_traducccion.py` parsea la respuesta y **valida 1:1** (si el conteo o la numeración no cuadran, no escribe nada). Así se tradujo *Die Verwandlung* de Kafka (10 partes, 864 oraciones, todas validadas).

5.2 Vía MT offline (opus-mt) — automática, sin API

Alternativa totalmente automática con **opus-mt** (Helsinki-NLP, modelos Marian) vía `transformers`, traducción directa **sin pivote**. `mt.py` está parametrizado por idioma de origen (`opus-mt-de-es` y `opus-mt-en-es`) y `traducir_mt.py` deduce el origen del cuerpo de cada lectura. La alineación 1:1 es automática (una salida por frase). Antes se descartó **Argos Translate**, que no tiene `de->es` directo y **pivota por inglés**, con errores graves. Con MT se tradujeron *Immensee* (`de->es`) y los **cuatro libros ingleses** — *The Time Machine* (Wells), *A Christmas Carol* (Dickens), *The Fall of the House of Usher* (Poe) y *Death in Venice* (Mann, trad. Kenneth Burke 1925) — 58 partes `en->es` sin copia-pegar. El par `en->es` es de los mejor entrenados de Helsinki-NLP, con calidad notablemente superior al `de->es`.

5.3 Comparación de calidad

Frase alemana	Gemini	opus-mt (directo)	Argos (pivote)
...einem ungeheueren Ungeziefer verwandelt	convertido en un monstruoso insecto	convirtiéndose en una bestia monstruosa	monstruoso vértigo (mal)
bestäubt (polvoriento)	cubiertos de polvo	polinizados (mal)	—
wohlgekleideter Mann	hombre bien vestido	vestido y vestido (mal)	—

Conclusión práctica: la MT offline **automatiza todo** el flujo pero comete errores semánticos que un LLM no comete. Para libros donde importe la calidad, conviene el LLM; para volumen o cuando la calidad “suficiente” basta, la MT. Ambas producen JSON estático, así que la app sigue siendo offline.

6. Errores encontrados y soluciones

Esta sección recoge los tropiezos del proceso; es la parte más útil para aprender, porque muchos son problemas de entorno o de comprensión de las librerías, no de la lógica.

ERROR / SOLUCIÓN — Certificados SSL en git (y luego en Python)

Síntoma: `git push` y las descargas de Python fallaban con “unable to get local issuer certificate”.

Causa: el entorno intercepta TLS y el almacén de certificados de OpenSSL/git no tenía el certificado raíz.

Solución: en git, `http.sslBackend=schannel` (usa el almacén de Windows, que se mantiene actualizado solo). En Python, instalar `pip-system-certs`, que hace que `requests` use también el almacén de Windows (necesario para descargar los modelos de spaCy, los diccionarios y los modelos de traducción).

ERROR / SOLUCIÓN — Reglas de los Hooks de React

Síntoma: riesgo de romper el orden de los hooks en `TableroClave`, que tenía un `return` de error antes de los `useState`.

Solución: mover **todos** los hooks al principio del componente y dejar los `return` condicionales después. Los hooks deben ejecutarse siempre en el mismo orden en cada `render`.

ERROR / SOLUCIÓN — `useEffect` disparado por un “gatillo falso”

Síntoma: para re-generar el tablero al cambiar de ronda se incrementaba un estado `numeroRonda` solo para forzar el `useEffect`.

Solución: extraer una función `iniciarPartida()` llamada explícitamente (en el montaje y al pasar de ronda). Intención explícita en vez de estado artificial.

ERROR / SOLUCIÓN — spaCy sobre-segmentaba las frases

Síntoma: *Die Verwandlung* daba 2.666 “frases” (Kafka usa frases largas; eran fragmentos).

Causa: el texto de Gutenberg viene con **líneas envueltas** (un salto de línea cada ~70 caracteres); spaCy segmentaba por esos saltos.

Solución: unir todas las líneas en un flujo continuo antes de segmentar (quitando además los marcadores de capítulo). Resultado: 864 frases reales.

ERROR / SOLUCIÓN — Compuestos y verbos fuertes sin traducción

Síntoma: ~19% de palabras sin traducción; muchos compuestos (*Musterkollektion*) y formas verbales (*hing*, *dasaß*).

Solución: cadena `de->en->es` con el enorme diccionario `deu-eng`, descomposición de la cabeza del compuesto, y modelo spaCy mediano para mejores lemas. Cobertura 70% -> 92%. Los verbos fuertes restantes exigirían un analizador morfológico más potente (rendimientos decrecientes).

ERROR / SOLUCIÓN — MT por pivote confundía el sentido

Síntoma: Argos traducía “Ungeziefer” (insecto) como “vértigo”.

Causa: Argos no tiene `de->es` directo; pivota por inglés y el doble salto acumula errores de sentido.

Solución: usar `opus-mt` directo `de->es` (Helsinki-NLP), que no pivota y da traducciones sin ese tipo de error grave.

ERROR / SOLUCIÓN — El lematizador español tropieza con arcaísmos y enclíticas

Síntoma: al glosar los libros del Siglo de Oro, spaCy producía lemas inexistentes: *celada* -> “celado”, *azotes* -> “azot”, y formas con pronombre enclítico (*preguntóle*, *trujeron*) quedaban sin analizar.

Causa: `es_core_news_md` está entrenado con español contemporáneo; el español áureo queda fuera de distribución.

Solución: el glosario casa por **forma superficial además de por lema**: una entrada como “trujeron” se define por su forma exacta y no depende del lematizador. Las voces del glosario se detectan con la frecuencia Zipf del español general (`wordfreq`), no con el análisis morfológico.

ERROR / SOLUCIÓN — Codificación de consola y shell en Windows

Síntoma: los umlauts salían como cajas o daban `UnicodeEncodeError (cp1252)`; y en cierto momento el shell de Bash se quedó sin PATH (ni `ls` funcionaba).

Solución: ejecutar Python con `PYTHONUTF8=1` (fuerza UTF-8 en E/S) y escribir siempre el JSON con `ensure_ascii=False`; y cambiar a PowerShell cuando Bash falló. Los datos siempre estuvieron bien (UTF-8); era solo la **visualización**.

ERROR / SOLUCIÓN — Imágenes enormes (rendimiento)

Síntoma: la imagen del hero pesaba **19 MB** (5192x3072).

Solución: redimensionar a 1600 px y convertir a WebP con `sharp`: 19 MB -> 46 KB (-99,75%). También se redujo el favicon (PNG de 200 KB -> 19 KB) y se corrigió su declaración `type`.

7. Ventajas y desventajas de cada implementación

7.1 Traducción por oración: LLM (Gemini) vs MT offline (opus-mt)

Criterio	LLM (Gemini)	MT offline (opus-mt)
Calidad	Muy alta, literaria	Buena, con errores semánticos
Automatización	Semi-manual (copia-pegar)	Total (script)
Coste / dependencia	Requiere el LLM (aunque offline: 1 vez)	Modelo local ~300 MB; CPU
Alineación 1:1	Hay que validarla (prompt + verificador)	Automática (una salida por frase)
Escalabilidad	Limitada por el copia-pegar	Alta (miles de frases)

7.2 Traducción por palabra: capas del traductor

Capa	Ventaja	Desventaja
Directo deu-spa	Mejor calidad y precisión	Cobertura limitada (~70%)
Cadena de->en->es	Gran cobertura (deu-eng es enorme)	Dos saltos: ruido y pérdida de sentido; se marca
Cabeza de compuesto	Recupera compuestos ausentes	Aproximada; puede errar (Handschuh)

7.3 Otras decisiones

Decisión	Ventaja	Desventaja
Texto paralelo para traducir la frase	Determinista, sin API; sirve para gramática futura	Sólo frase, no palabra; hay que alinear

Decisión	Ventaja	Desventaja
Léxico para traducir la palabra	Traducción por palabra offline y precisa	No capta contexto; requiere el pipeline
localStorage para el estado	Cero backend; privado; simple	No sincroniza entre dispositivos
Modelo spaCy md vs sm	Mejor lematización (etiquetado más preciso)	Más peso y algo más lento
Léxico por dynamic import	Bundle inicial mucho menor; cacheado	Pequeña latencia al abrir la primera lectura

8. Librerías, modelos y recursos

8.1 Frontend

Recurso	Licencia	Uso
React 19 / react-dom	MIT	UI por componentes
Vite 8	MIT	Bundler + dev server
react-router-dom 7	MIT	Enrutado y URLs
Vitest 4	MIT	Pruebas del motor puro
sharp (dev, puntual)	Apache-2.0	Optimización de imágenes (WebP)

8.2 Pipeline de PLN

Recurso	Licencia	Uso
spaCy + de_core_news_md / en_core_web_sm / es_core_news_md	MIT	Segmentación, tokenización, lematización, dependencias (un modelo por idioma)
FreeDict deu-spa / deu-eng / eng-spa (TEI)	Libre (GPL/CC)	Traducción por palabra (directo + cadena; el inglés va directo por eng-spa)
transformers + torch	Apache-2.0 / BSD	Ejecutar opus-mt
Helsinki-NLP/opus-mt-de-es y opus-mt-en-es (Marian)	CC-BY / Apache	MT por frase de->es y en->es (offline)
wordfreq	Apache-2.0	Frecuencia Zipf del español general: detecta las voces raras a glosar
glosas_es.json (curado propio)	—	Glosario monolingüe de voces raras del español (definiciones a mano)
Argos Translate (evaluado, descartado)	MIT/CC	MT por pivote; peor calidad para de->es
pip-system-certs	BSD	Usar el almacén de certificados de Windows (SSL)
Project Gutenberg (textos)	Dominio público	Corpus de lectura (Kafka, Storm, Wells, Dickens, Poe, Mann, Cervantes, Quevedo, Darío, Quiroga, Esopo, Grimm)

8.3 Modelos: notas de tamaño y calidad

- **de_core_news_md / es_core_news_md**: modelos medianos; mejor etiquetado y lematización que los pequeños. Clave para reducir errores de lema.
- **opus-mt-de-es / opus-mt-en-es**: ~300 MB cada uno; inferencia en CPU; directos (sin pivote). El par en->es da calidad claramente superior.

- Las dependencias pesadas de MT (transformers/torch) y los modelos **no** se versionan en git; se documentan y se cachean fuera del repo. Solo se versiona el JSON de salida.

9. Insights y metodología

- **Separar lo puro de lo impuro.** Extraer la lógica (LCG, bolsa, progreso) a módulos sin efectos secundarios permitió probarla con Vitest y reutilizarla; el patrón se repitió en el pipeline (traductor, construir_lexico).
- **Medir antes de optimizar.** El diagnóstico de cobertura por POS reveló que el problema no era la lematización sino el **tamaño del diccionario**; eso orientó la solución (cadena por inglés) en vez de perseguir mejoras marginales del lematizador.
- **Validar la alineación siempre.** El verificador 1:1 del importador de traducciones convirtió un paso frágil (pegar la salida de un LLM) en algo seguro: si algo no cuadra, no escribe nada.
- **Formatos “pipeline-ready”.** Diseñar el JSON desde el principio con el formato que emitiría el pipeline evitó rehacer el frontend cuando este llegó.
- **Diferenciar problema de datos y problema de visualización.** Muchos “errores” (umlauts como cajas) eran solo codificación de consola; los datos estaban bien.
- **Procedencia y licencias.** Cada lectura guarda su campo fuente con la obra, el autor y la URL de dominio público: transparencia para la tesis.

10. Estado actual y trabajo futuro

Hecho

- Sección de Lectura completa (biblioteca, lector, bolsa, progreso, libros por partes) en **tres idiomas de estudio**.
- Pipeline de PLN multilingüe: ingesta con modelo spaCy por idioma (y fragmentos con max_chars/fin), traducción por palabra (FreeDict por capas para de; eng-spa directo para en), por frase (Gemini y opus-mt de->es / en->es) y **glosado monolingüe del español** (wordfreq + glosario curado).
- Contenido: 9 lecturas cortas trilingües + 2 libros alemanes (Die Verwandlung, Immensee) + 4 ingleses (Wells, Dickens, Poe, Mann) + 4 españoles (Quijote y Buscón en fragmento; Azul... y los Cuentos de Quiroga completos).
- Idioma de estudio global (contexto persistido), bolsa filtrada por idioma y perfil export/import sin servidor.
- Motor puro con pruebas (161 en toda la plataforma); Codenames refactorizado; recursos optimizados.

Pendiente / próximos pasos

- **Ampliar el glosario español:** subir la densidad de glosas de Azul/Quiroga y extender el fragmento del Quijote (p. ej. hasta los molinos de viento, cap. VIII).
- **Léxico por token:** anotar la traducción por posición en el pipeline para resolver la ambigüedad de forma general (hoy: overrides por lectura).
- **Optimización adicional del peso:** separar metadatos del contenido para cargar las lecturas también bajo demanda; dividir el léxico por idioma/libro.
- **Mejorar cobertura y MT:** analizador morfológico para verbos fuertes alemanes; evaluar modelos de MT mayores.

11. Glosario técnico

Término	Definición
PLN / NLP	Procesamiento de Lenguaje Natural.
spaCy	Librería de PLN en Python (segmentación, POS, lemas, dependencias).
Lema / lematización	Forma de diccionario de una palabra (kauft -> kaufen).
POS	Part-of-speech: categoría gramatical (VERB, NOUN, ADJ...).
Dependencia svp	Etiqueta de spaCy para el prefijo de un verbo separable alemán.
Verbo separable	Verbo alemán cuyo prefijo se desplaza (bereitet ... vor = vorbereiten).
FreeDict	Colección de diccionarios bilingües libres en formato TEI (XML).
TEI	Text Encoding Initiative: estándar XML para textos y diccionarios.
MT	Machine Translation (traducción automática).
opus-mt / Marian	Modelos de MT abiertos de Helsinki-NLP.
LLM	Large Language Model (p. ej. Gemini).
Alineación 1:1	Correspondencia frase a frase entre el original y su traducción.
LCG	Linear Congruential Generator: generador pseudoaleatorio (aritmética modular).
Fisher-Yates	Algoritmo para barajar un arreglo de forma uniforme.
SPA / bundle	Single Page Application; bundle: JS empaquetado que se descarga.
dynamic import	Carga de un módulo bajo demanda; genera un chunk aparte en Vite.
Cobertura de traducción	% de formas de palabra distintas que reciben traducción.
Glosa / glosado	Definición breve en la misma lengua para una voz rara (español).
Escala Zipf	log10 de la frecuencia por millón de palabras; mide la rareza léxica.
wordfreq	Biblioteca Python con frecuencias léxicas multilíngües (escala Zipf).
Leitner / SM-2 / FSRS	Algoritmos de repetición espaciada para el repaso.

Documento generado con ReportLab a partir del historial de desarrollo del proyecto *indescifrable*. Repositorio: github.com/Eduard-Cini/indescifrable.